



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: ALFONSO D'HERNÁN RODRÍGUEZ
ZULUAGA

N° de Cuenta: 32156155-6

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 17 de marzo del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.-

a) Ejercicio de completado de la cabina.

Código modificado:

Window.h

```
GLfloat getrueda1() { return rueda1; }  
GLfloat getrueda2() { return rueda2; }  
GLfloat getrueda3() { return rueda3; }  
GLfloat getrueda4() { return rueda4; }  
  
GLfloat rueda1, rueda2, rueda3, rueda4;
```

Window.cpp

```
rueda1 = 0.0f;  
rueda2 = 0.0f;  
rueda3 = 0.0f;  
rueda4 = 0.0f;  
  
if (key == GLFW_KEY_U)  
    theWindow->rueda1 += 10.0;  
  
if (key == GLFW_KEY_I)  
    theWindow->rueda2 += 10.0;  
  
if (key == GLFW_KEY_O)  
    theWindow->rueda3 += 10.0;  
  
if (key == GLFW_KEY_P)  
    theWindow->rueda4 += 10.0;
```

Main

```
// Base

model = modelaux_2;
model = glm::translate(model, glm::vec3(0.0f, -3.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(8.0f, 2.0f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[4]->RenderMesh(); //dibuja cubo y pirámide triangular

// Rueda 1

model = modelaux;
model = glm::translate(model, glm::vec3(-4.0f, -1.0f, -2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrueda1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMesh();

// Rueda 2

model = modelaux;
model = glm::translate(model, glm::vec3(4.0f, -1.0f, -2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrueda2()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMesh();

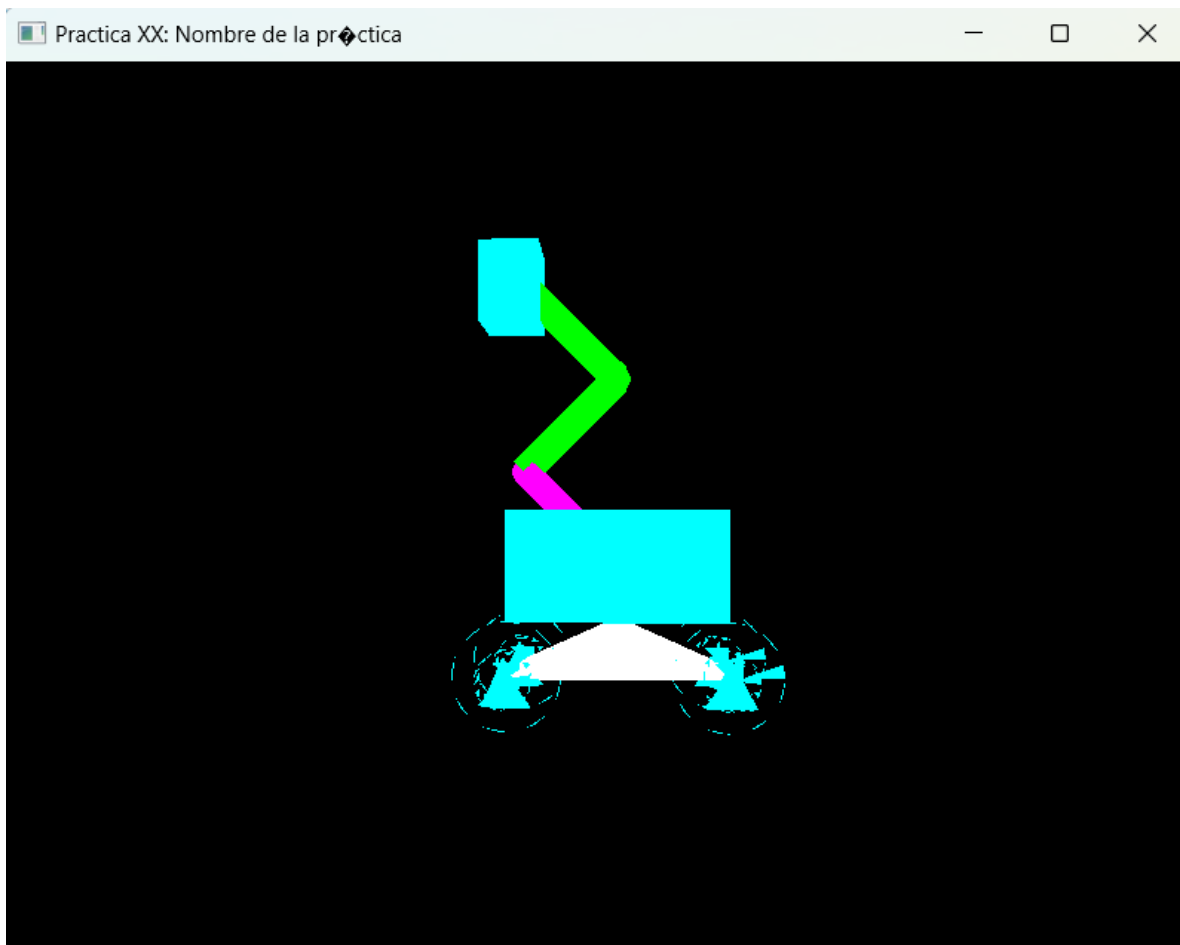
// Rueda 3

model = modelaux;
model = glm::translate(model, glm::vec3(4.0f, -1.0f, 2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrueda3()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMesh();
```

```
// Rueda 4
```

```
model = modelaux;  
model = glm::translate(model, glm::vec3(-4.0f, -1.0f, 2.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getrueda4()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));  
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(0.0f, 1.0f, 1.0f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos  
meshList[2]->RenderMesh();
```

Resultado final:



b) Ejercicio del animal mecánico

Código

Window.h

```
GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6, articulacion7, articulacion8;  
GLfloat rueda1, rueda2, rueda3, rueda4;  
GLfloat cola, cola_sin;
```

Window.cpp

```
if (key == GLFW_KEY_N) {
    theWindow->articulacion7 += 10.0;
}
if (key == GLFW_KEY_M) {
    theWindow->articulacion8 += 10.0;
}
if (key == GLFW_KEY_T) {
    theWindow->cola_sin += 0.4;
    theWindow->cola = 5*sin(theWindow->cola_sin);
}
```

Main

```
// Torso
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 6.0f, 4.0f));
modelaux = model;
modelaux_2 = model;
model = glm::scale(model, glm::vec3(8.0f, 4.0f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.647f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh();
model = modelaux;

// Pierna superior frontal izquierda
model = glm::translate(model, glm::vec3(1.75f, -0.5f, -2.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.25f, 2.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.647f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh();

model = modelaux;
```

```

// Articulación 1 frontal izquierda
model = glm::translate(model, glm::vec3(0.0f, -1.25f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(300.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.647f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

model = modelaux;

// Pierna inferior frontal izquierda
model = glm::translate(model, glm::vec3(0.9f, 0.00f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.8f, 0.95f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.647f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh();

model = modelaux;

// Articulación 2 frontal izquierda
model = glm::translate(model, glm::vec3(0.9f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(60.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.647f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

model = modelaux;

// Pata frontal izquierda
model = glm::translate(model, glm::vec3(0.7f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.4f, 0.5f, 1.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.647f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh();

model = modelaux_2;

```

(Este código se repite 3 veces para el resto de las extremidades, con la única diferencia yaciendo en las coordenadas.)

```

model = modelaux_2;

// cabeza

model = glm::translate(model, glm::vec3(4.0f, 2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.647f, 0.0f);
sp.render();

model = modelaux;

// hocico

model = glm::translate(model, glm::vec3(1.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(2.0f, 1.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.647f, 0.0f);
meshList[2]->RenderMesh();

```

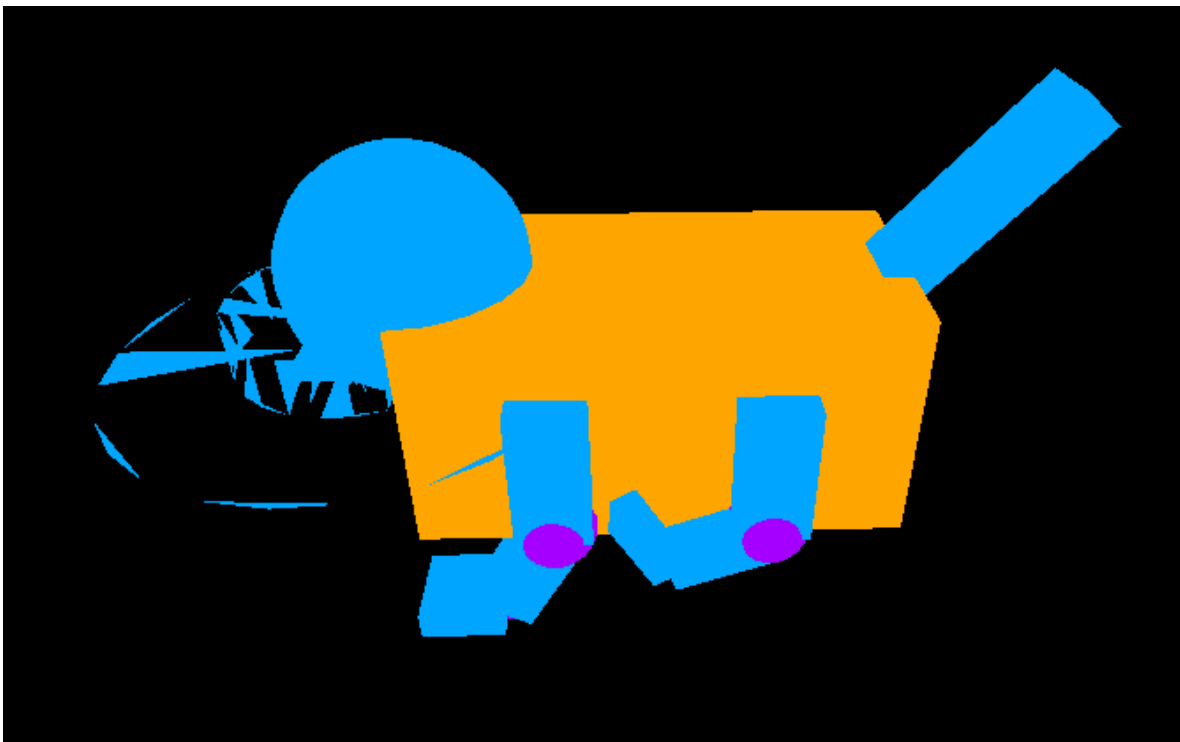
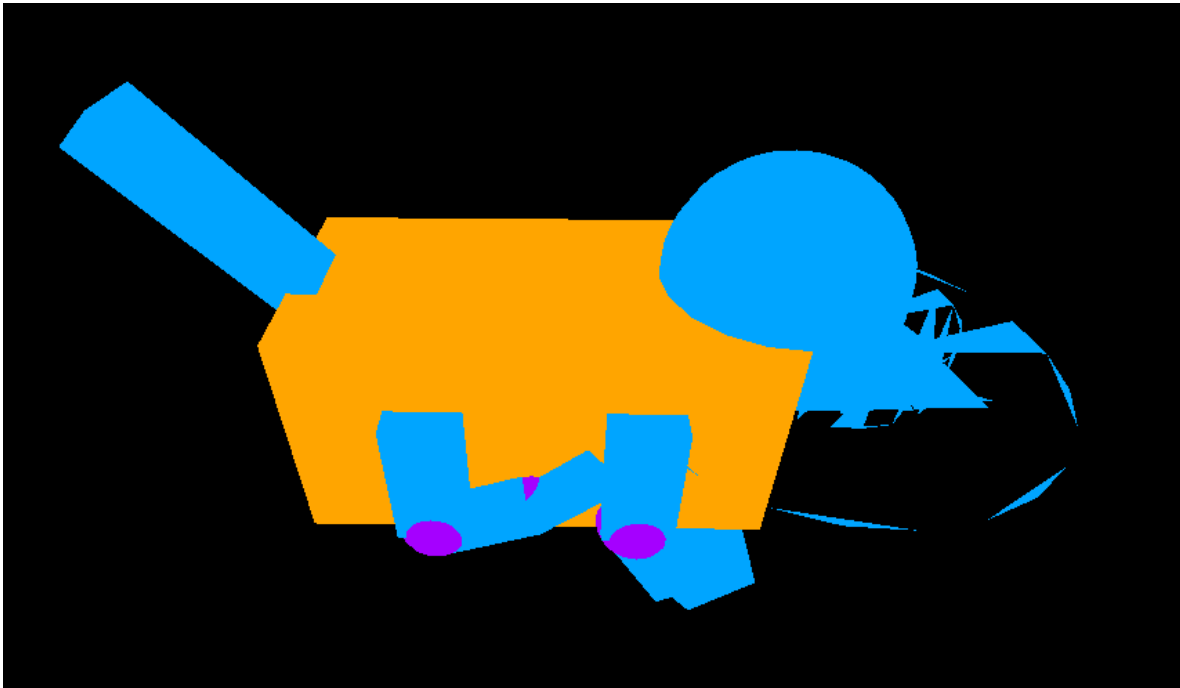
```

model = modelaux_2;

//cola
model = glm::translate(model, glm::vec3(-4.0f, 2.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getcola()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(7.0f, 0.75f, 1.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.647f, 0.0f);
meshList[0]->RenderMesh();

```

Resultado



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Tuve unos cuantos problemas a lo largo de la llevada a cabo tanto de la práctica como del ejercicio de clase para el cambio de colores, pues algo ocurría en donde

simplemente cambiaba el color de todos los modelos concatenados hasta el punto más reciente. Lo resolví mirando el código.

- Los cilindros siguen rotos. Sigo sin entender la problemática, quiero pensar que es una cuestión de mi computadora.

- Para que la cola no solamente rotara en su mismo lugar, hice uso de una función seno para limitar el rango dentro del que se mueve la cola.

3.- Conclusión:

Ninguno de los dos ejercicios fue particularmente difícil, potencialmente el primero lo fue debido a que la planificación de estos mismos fue particularmente fácil. La implementación se complicó un poco pero fuera de esto no fue nada imposible.

No hay comentarios generales.

Debido a que llegué tarde a la clase, debo de mencionar que al principio no entendía muy bien la metodología deseada para cada uno de los objetos creados. Los diagramas hechos en clase no me quedaban claros hasta que empecé a tirar código, de manera que puedo imaginar perfectamente la metodología que se tiene que llevar a cabo para poder transformar los diferentes objetos a lo largo del plano propuesto. Con esto en mente, pude fácilmente llevar a cabo cada una de las jerarquías necesarias para implementar ambos ejercicios. Como el objetivo de la práctica es implementar y entender el modelado jerárquico para el modelado más complicado de ciertas figuras, podemos considerar esta práctica como satisfactoriamente concluida.

Bibliografía en formato APA

Román Guadarrama, J. R. (2026). Manual de prácticas: (Práctica No. 03). Universidad Nacional Autónoma de México, Facultad de Ingeniería, División de Ingeniería Eléctrica, Departamento de Computación.